



NIMH MonkeyLogic: Behavioral control and data acquisition in MATLAB

Jaewon Hwang*, Andrew R. Mitz, Elisabeth A. Murray

Section on Neurobiology of Learning and Memory, Laboratory of Neuropsychology, National Institute of Mental Health, National Institutes of Health, Bethesda, MD 20892, USA

ARTICLE INFO

Keywords:

Neurophysiology
Psychophysics
Software
Cognition
Human
Monkey

ABSTRACT

Background: Computerized control of behavioral paradigms is an essential element of neurobehavioral studies, especially physiological recording studies that require sub-millisecond precision. Few software solutions provide a simple, flexible environment to create and run these applications. MonkeyLogic, a MATLAB-based package, was developed to meet these needs, but faces a performance crisis and obsolescence due to changes in MATLAB itself.

New method: Here we report a complete redesign and rewrite of MonkeyLogic, now NIMH MonkeyLogic, that natively supports the latest 64-bit MATLAB on the Windows platform. Major layers of the underlying real-time hardware control were removed and replaced by custom toolboxes: NIMH DAQ Toolbox and MonkeyLogic Graphics Library. The redesign resolves undesirable delays in data transfers and limitations in graphics capabilities.

Results: NIMH MonkeyLogic is essentially a new product. It provides a powerful new scripting framework, has dramatic speed enhancements and provides major new graphics abilities.

Comparison with existing method: NIMH MonkeyLogic is fully backward compatible with earlier task scripts, but with better temporal precision. It provides more input device options, superior graphics and a new real-time closed-loop programming model. Because NIMH MonkeyLogic requires no commercial toolbox and has a reduced hardware requirement, implementation costs are substantially reduced.

Conclusion: NIMH MonkeyLogic is a versatile, powerful, up-to-date tool for controlling a wide range of experiments. It is freely available from <https://monkeylogic.nimh.nih.gov/>.

1. Introduction

Some of the earliest computers developed for the laboratory were used in studies of behavior and physiology (Rosenfeld, 1983). Advances in computer engineering and information technology continue to drive laboratory implementation of behavioral and physiological experiments, especially in studies that required precise control of stimulus presentation and data collection with sub-millisecond temporal precision. General purpose data acquisition and control systems used by engineers require knowledge and programming skills not common among basic science researchers. In response, software tools have been developed to aid researchers with task design, stimulus presentation and data acquisition. A good example is NIMH Cortex (Laboratory of Neuropsychology / NIMH / NIH, 1989), a behavioral control and data acquisition system developed in 1989 that was widely used by primate behavioral neurophysiologists. As is the case for similar early systems, operating system evolution and computer hardware advancements have led to its gradual abandonment.

MonkeyLogic (ML1) is a MATLAB-based toolbox for designing and executing psychophysical tasks. Developed a decade ago (Asaad et al., 2012; Asaad and Eskandar, 2008a), ML1 was widely adopted as a replacement for NIMH Cortex. ML1 inherited elements from NIMH Cortex, such as the “conditions” and “state” files but moved away from the need to master the C/C++ programming language. ML1 uses MATLAB, a higher level language and one commonly used in neuroscience research for data analysis. Like NIMH Cortex, ML1 can present stimuli and store analog and event data. Based on the frequent occurrence of ML1 citations in peer-reviewed journal articles, ML1 rose in popularity among nonhuman primate behavioral researchers soon after its introduction. However, ML1 is approaching obsolescence. It is not compatible with the latest versions of MATLAB due to Mathworks’s roadmap goals of supporting a new graphics system and only 64-bit environments. ML1 is not compatible with the new HG2 graphics engine introduced in MATLAB R2014b. It also depends on the legacy 32-bit interface of the MATLAB Data Acquisition (DAQ) Toolbox, which was discontinued from MATLAB R2016a.

* Corresponding author at: Laboratory of Neuropsychology, NIMH, 49 Convent Drive, Room 1B80, Bethesda, MD 20892-4415, USA.

E-mail address: jaewon.hwang@nih.gov (J. Hwang).

<https://doi.org/10.1016/j.jneumeth.2019.05.002>

Received 19 December 2018; Received in revised form 2 May 2019; Accepted 5 May 2019

Available online 06 May 2019

0165-0270/ Published by Elsevier B.V.

To protect the user experience and knowledge with ML1 programming and ensure the long-term availability of this useful research tool, we developed a new MATLAB toolbox: NIMH MonkeyLogic (ML2). ML2 is complete rewrite of ML1. It is object-oriented and fully supports the latest 64-bit MATLAB. In addition to backward compatibility with ML1 scripts, ML2 provides a new, powerful scripting framework that enables users to create dynamic, behavior-responsive stimuli and to track complex behavior patterns online. ML2 is equipped with a new data acquisition tool (NIMH DAQ Toolbox) and a new graphics library (MonkeyLogic Graphics Library). These custom-built components not only resolve the drawbacks associated with the previous generation of tools, but they also support more input devices and more stimulus types. The new toolboxes reduce the number of commercial software products required and remove the earlier requirement for using two input/output interface cards for real-time performance. Together, ML2 changes lower the financial bar for implementing a behavioral control and recording system. This report describes key issues resolved in ML2 and the performance results acquired from benchmark tests.

ML2 can be used to program various sensory, motor or cognitive tasks for human or animal subjects. It is suitable for the implementation of any behavioral paradigm that requires the controlled presentation of a variety of stimuli and the acquisition of responses with high temporal precision. Typical examples are visual receptive field mapping, object tracking by eye or arm movements, preferential viewing, oculomotor delayed-response tasks, delayed match-to-sample tasks, etc. Sample codes for these tasks can be found in the ML2 distribution packages, which are downloadable from the ML2 website (<https://monkeylogic.nimh.nih.gov/>). The packages also includes detailed manuals that explain how to set up ML2 and write tasks with it.

2. Materials and methods

The system used for testing was a DELL OptiPlex 7050 (Dell Inc., Round Rock, TX, USA) with an Intel i5-7600 CPU running at 3.50 GHz, 16GB of RAM and a 1TB solid state drive. The operating system (OS) was 64-bit Windows 10 professional (Microsoft, Redmond, WA, USA). One AMD Radeon R5 430 graphic board (Advanced Micro Devices, Santa Clara, CA, USA) with 2 GB of video memory was installed that supported one DisplayPort and one DVI port. A 32-inch Samsung U32H850 monitor running at 3840 × 2160 resolution and 60 Hz was used as the experimenter's display. For the subject's display, a 17-inch Dell P793 CRT monitor running at 1024 × 768 resolution and 75 Hz and other LCD monitors (see Table 2) were tested. MATLAB R2014a 32 bit (The Mathworks Inc., Natick, MA, USA) is the last version that can run ML1 (version: 4-05-2014 build 1.0.26) without any compatibility issue and was used for most of the tests to compare the performance of ML1 and ML2 (version: Jun 11, 2018 build 139) under matched conditions. R2014a was installed with the MATLAB Data Acquisition Toolbox and the Image Processing Toolbox, to run all aspects of ML1. ML2 does not use those two toolboxes. ML2 was also tested with R2018a 64 bit to measure performance using the latest MATLAB version. Java Virtual Machine was enabled for all tests.

Behavioral signals were recorded using either MATLAB DAQ Toolbox with two National Instruments ("NI") PCIe-6323 boards for ML1 (National Instruments, Austin, TX, USA) or NIMH DAQ Toolbox with one NI PCIe-6323 board for ML2. As described later, NIMH DAQ Toolbox does not have the earlier problem of slow sample transfers, even when logging and sampling data from a single interface card, and therefore does not require two NI boards for real-time behavior monitoring. Two NI USB-6008 devices were also tested to compare the performance of USB to PCIe bus types. Eye gaze signals provided via an Arrington ViewPoint EyeTracker (Arrington Research, Inc., Scottsdale, AZ, USA) at 220 Hz were used for the test described below. Both ML1 and ML2 were set to send out 16-bit event markers through digital output lines of the NI boards. For the detection of the visual stimulus onset, a photodiode was attached to the top-left corner of the subject's

screen. Analog stimulations were delivered through analog output channels of the same NI boards. Sounds were played via an on-board audio codec (Realtek Audio; Realtek Semiconductor Corp., Hsinchu, Taiwan) or a USB stereo sound adapter (SD-CM-UAUD; Syba Multimedia Inc., Chino, CA, USA).

We evaluated the general performance of ML2 in creating stimuli and storing data, as these are two basic functions of all behavioral control software. The behavioral task used for performance testing went as follows (the task code is available with ML2 distribution packages). A trial began with a fixation point presented at the center of screen. After a 1-s fixation period, a random dot motion stimulus (dot size: 0.1°, aperture size: 5°, dot density: 95.5 dots/degree²/s) was presented for 1 s. Its direction was 0 or 180° and the coherence level was either 25 or 40 percent. Next, two choice targets were presented on the left and right sides of the screen. The choice was indicated by gaze. A correct choice triggered an external reward device. In ML2, the random dot stimulus can be created from a movie file, generated on-line from movie data, or drawn directly frame by frame. Also, there is an option to reuse the created movie object without reloading it. The task was run multiple times with each option and average performance was calculated separately for each option.

During performance tests, the task was run at the "high process priority" class of Windows, which does not yield the processor time to other applications frequently. In Windows 7 and later, high process priority is the highest class allowed without invoking administrator privileges to run MATLAB. Switching between high process priority and normal priority classes is done automatically by MonkeyLogic, when a task starts and ends.

Additional benchmark tests were performed to measure the lags in triggering external devices and presenting stimuli. To estimate event marker delays, a digital output was wired to an analog input of the same board. The analog input was monitored at 40 kHz and the issuance time of the event marker commands was subtracted from the time of the resultant analog input signal change. Event markers also provided a synchronizing signal for measuring stimulus latencies. The stimulus latency is the time between an event marker sent from a stimulus presentation command and a stimulus (visual object, sound and stimulation) presented at the target devices (monitor, speaker and analog output). To make this measurement, the event marker output and the output of the target device were recorded simultaneously at 40 kHz on another computer and their onset times were compared.

All tasks were written as simple m-scripts and they are available with NIMH MonkeyLogic distribution packages from the NIMH MonkeyLogic website (<https://monkeylogic.nimh.nih.gov/>) or upon request.

3. Results

We first introduce the new scripting framework of ML2 and explain advanced features of its components: NIMH DAQ Toolbox and MonkeyLogic Graphics Library. Then, we discuss ML2's temporal performance with the test results.

3.1. A new scripting method: the scene framework

The ability to generate stimuli that change progressively, or in response to behavioral input, is important for designing interactive and feedback-rich experiments. ML1 is designed for programming simplicity, but presenting dynamic, behavior-responsive stimuli is structurally difficult and often inefficient. ML1 processes stimulus presentation and behavior tracking using two separate functions, `toggleobject()` and `eyejoytrack()`, respectively (Fig. 1A). The stimuli presented by `toggleobject()` cannot be manipulated while `eyejoytrack()` is running. In addition, ML1 requires iterating a loop at millisecond intervals during `eyejoytrack()` to fetch and process each new data sample. This frequent loop iteration does not allow sufficient time to perform sophisticated

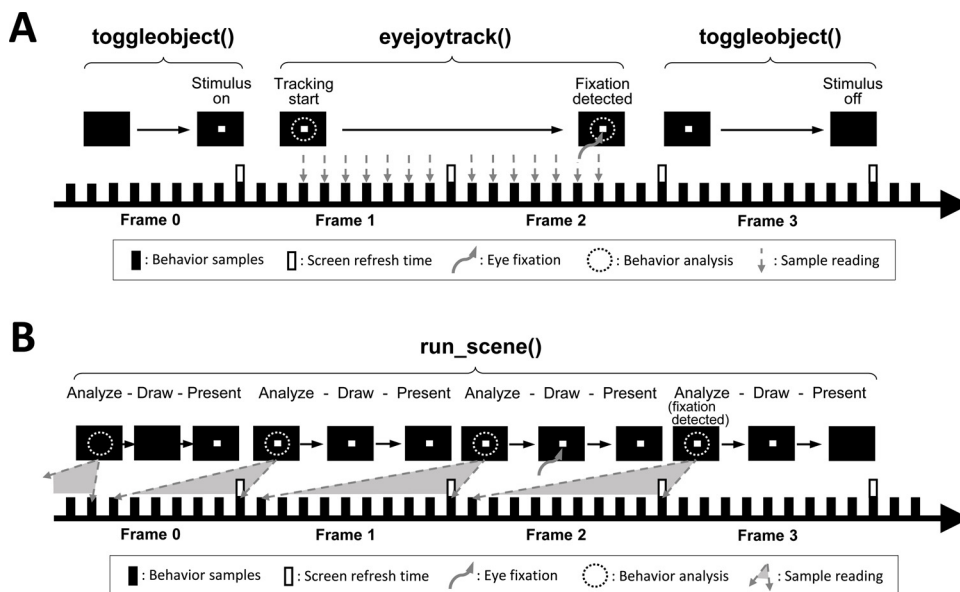


Fig. 1. Detection of eye fixation in the previous scripting method and the scene framework. **A.** In the previous scripting method, the stimulus (fixation point) is turned on and off by toggleobject() and the target behavior (eye fixation) is monitored by eyejoytrack(). During eyejoytrack(), behavior samples are checked every millisecond. eyejoytrack() ends as soon as the fixation is detected, but the fixation point may not be turned off until the end of the next frame. **B.** In the scene framework, the presentation of the fixation point and the detection of eye fixation are processed in parallel by run_scene(). run_scene() repeats a cycle of analyzing behavior samples, drawing visual stimuli on a background buffer and presenting the buffer every frame. All the samples collected during the previous frame are analyzed together at the beginning of each frame. Therefore, the occurrence of the fixation may not be known until the next frame, although the time of the fixation can still be calculated online accurately.

computations or draw complex stimuli between video frames. To resolve these issues, ML2 retains the functions in ML1, but also adds a new scripting framework (“scene framework”), in which both stimulus presentation and behavior tracking are handled by one function, run_scene() (Fig. 1B). In this framework, behavior samples collected during one frame interval (16–17 samples at a 60-Hz refresh rate during 1-kHz data acquisition) are analyzed together at the beginning of the next frame interval and the screen is redrawn in time for the next frame. Therefore, a cycle of sample analysis, screen drawing and presenting is repeated each frame.

Fig. 2 shows an example task, Fixation Timer, written with the scene framework. This task displays 7 annuli (“timer”) that are continuously filled up in the clockwise direction at the speed of one annulus per second from the right-most annulus to the leftmost one (Fig. 2A). The timer counts, however, only while gaze is maintained on the white square (“fixation point”) presented at the center of the screen. If the gaze moves out of the fixation window, time counting stops. Counting resumes if the gaze returns to the window within 4 s. This online stimulus manipulation tied closely to a behavior is possible because of the new scripting framework.

In the scene framework a task is built by assembling small blocks, called “adapters”, into scenes and running the scenes according to the task flow. Adapters are MATLAB class objects designed to detect a simple behavior pattern from input data or draw a basic geometric shape. Multiple adapters can be concatenated into a “chain” to do complex tasks. For the Fixation Timer task, a chain of three adapters, including EyeTracker, SingleTarget and TimerDemo, is used (Fig. 2B). The adapters have two member functions, analyze() and draw(), and they are called alternately by run_scene() for behavior analysis and screen update (Fig. 2C). The job of each adapter in the chain is modularized so that the upper-level adapter may perform a more complex job based on the lower-level adapter’s work. For example, the SingleTarget adapter determines fixations with eye signal samples that the EyeTracker adapter retrieves from the DAQ device. The TimerDemo adapter counts the fixation duration based on the fixation on/off status analyzed by the SingleTarget adapter.

The scene framework has several advantages over the previous scripting method. First, its frame-by-frame stimulus control facilitates the creation of dynamic stimuli, such as random dot kinematograms and moving gratings. Second, the adapters carry the benefits of object-oriented programming including encapsulation and code reuse. ML2 is distributed with more than 40 built-in adapters in the latest release. Users can create their own adapters from a provided template and share

these publicly. Third, the extendible adapter chains allow users to code complex tasks without many of the structural restrictions of the ML1 scripting structure. Despite its benefits, the scene framework does not completely replace the previous programming functionality. There are cases in which the synchronization of behavioral events with video frames is undesirable. For example, it may be necessary to immediately send an event marker on a behavioral event in some cases, without waiting until the next frame. Such an immediate response to an external event may be best coded using toggleobject() and eyejoytrack().

3.2. NIMH DAQ toolbox

A data acquisition device typically supports two different modes for analog data acquisition. In one mode, the device acquires a single sample and sends it directly to the user application. This one-sample mode (also called the nonlogging mode) is best for acquiring the most recent state of the input signal. In the other mode, data are streamed from the input device at regular intervals determined by an on-board clock signal. Streamed (also called logging) data are preferred for data analysis. The problem with streamed data is that it is often buffered, which can delay access to the most recent data point.

ML1 and other MATLAB-based behavior control tools that depend on MATLAB DAQ Toolbox for data acquisition read from the memory buffer every ~15 ms in the logging mode, even though the DAQ device stacks new samples to the buffer at a much faster rate (Asaad and Eskandar, 2008b). This long readout interval lowers the frequency of accessing new samples practically to ~67 Hz and prevents user applications from analyzing behavioral changes with millisecond resolution (Fig. 3A). To mitigate this problem, ML1 uses two DAQ boards that process the same signals: one running in the logging mode and storing samples for offline analysis and the other running in the (nonlogging) one-sample mode and providing the current state of behavioral input without buffering delay.

NIMH DAQ Toolbox obviates the problem of limited access to new samples while using the logging mode. It uses a flexible buffer structure and multithreaded functions that allow access to new samples at any time requested. As a result, new samples can be obtained at 1 kHz, or faster, even during the logging mode (Fig. 3B–C). We tested the ML2 sampling performance with up to 32 analog channels without missing the 1-ms deadline (Fig. 3D). Thus, NIMH DAQ Toolbox obviates the need for two DAQ boards.

USB DAQ devices work faster when used with NIMH DAQ Toolbox compared to the MATLAB DAQ Toolbox. NI USB-6008 using the

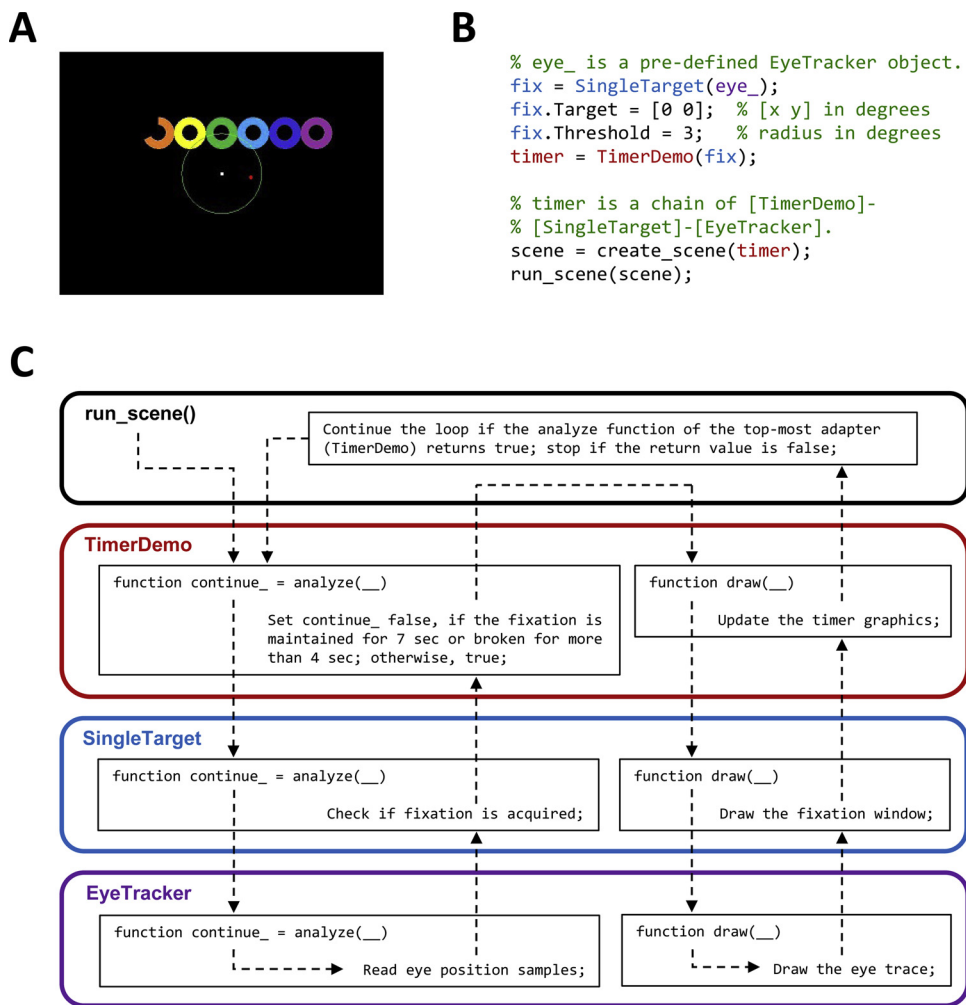


Fig. 2. An example task written with the scene framework. This is a working example that is executable in ML2. A video clip of this task and the complete task files are available at NIMH MonkeyLogic website. **A.** The fixation timer task. The total duration of fixation is indicated by the annulus-shaped counters that continuously change over time. The counter stops when the fixation is broken but resumes if the fixation is re-acquired within 4 s. **B.** The script for the fixation timer task. Three adapters are used and combined into a chain. Each adapter can be manipulated by changing its properties. The chain is executed via `create_scene()` and `run_scene()`. **C.** The cycle during `run_scene()`. Each adapter has two member functions: `analyze()` and `draw()`. `run_scene()` calls `analyze()` and `draw()` of the top-most adapter, the adapter given as an input argument to `create_scene()`, which triggers the iteration of all linked adapters. `run_scene()` repeats this cycle until `analyze()` of the top-most adapter returns false.

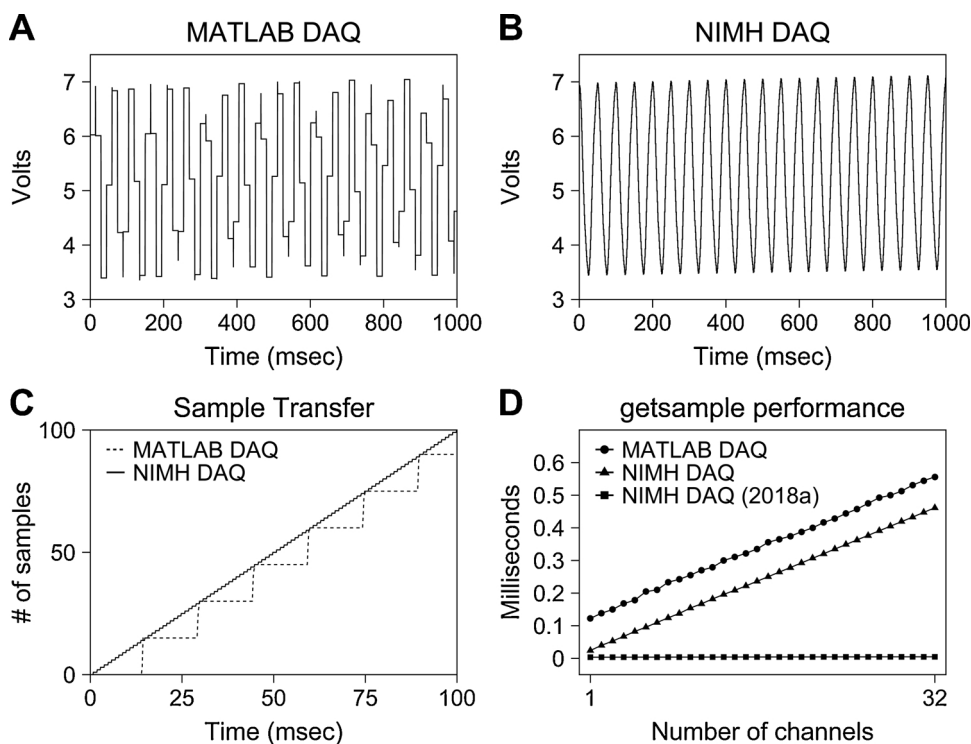


Fig. 3. Analog input recordings with MATLAB DAQ Toolbox and NIMH DAQ Toolbox. The input signal was a 20-Hz sine wave and the sampling rate was 1 kHz. **A.** Due to the delay in sample transfers, MATLAB DAQ can check new samples only at a rate of ~67 Hz during the logging-mode recordings, which results in aliasing (see the main text). The buffer size was adjusted so that the sample transfers could occur as frequently as possible. **B.** NIMH DAQ Toolbox retrieves new samples at any time requested even in the logging mode and shows no aliasing. **C.** The number of available samples increases by 1 every millisecond in NIMH DAQ Toolbox, whereas it increases by 15 every 15 ms in MATLAB DAQ Toolbox. **D.** The temporal cost of checking the current input state as a function of the number of channels.

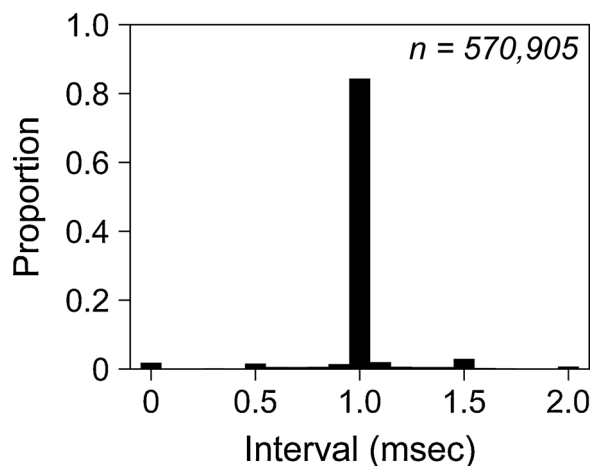


Fig. 4. Sampling intervals of the Windows software timer. The samples are collected for 100 trials. The bin size is 0.1 ms.

MATLAB DAQ Toolbox has a maximum collection rate of 360–380 Hz in the nonlogging mode, which is consistent with the general belief that USB devices are too slow for practical use with ML1. However, NIMH DAQ Toolbox retrieves unique samples from the USB-6008 at rates of 1.8–2.2 kHz. This fast sample retrieval makes it possible to assemble a ML2 system in a portable computer with USB devices.

NIMH DAQ Toolbox supports 1 kHz data acquisition from devices that do not produce analog signals, such as touchscreens, USB joysticks and TCP/IP eye trackers. Because the output of these devices cannot be processed via the DAQ board, the computer system needs to read their states at regular intervals on its own. NIMH DAQ Toolbox does this by using the software timer of the operating system (OS). The accuracy of the OS timer may not be as good as the hardware timer and can be affected by the workload of the system. Nevertheless, our performance test showed that the sample intervals were mostly 1 ms (0.99 ± 0.22 ms, mean $\pm$ SD), as intended (Fig. 4).

Time-stamped video capture is also supported by NIMH DAQ Toolbox and hence by ML2. This feature requires cameras with driver software that provides a DirectShow capture filter. Most webcams currently on the market meet this requirement. The details of capture capabilities, such as video size and color depth, may vary from camera to camera.

3.3. MonkeyLogic graphics library

ML1 provides icons on the experimenter's screen to indicate what items the subject sees and how the subject responds. ML1 also has a text window for task-related messages. ML2 provides a greatly enhanced environment for more realistic and detailed monitoring of the subject's experience through the implementation of the MonkeyLogic Graphics Library (MGL). MGL creates two parallel graphic screens: one for the subject and the other for the experimenter (Fig. 5). When visual stimuli are drawn on the subject's screen, MGL also draws exact replicas scaled to the experimenter's screen. Additional information for the experimenter, such as online states of input signals, fixation windows and custom user strings, is displayed on the experimenter's screen only. Managing two views takes additional computational resources. However, these tasks do not tax the abilities of modern video cards with multi-monitor support and hardware accelerated graphics. The rendering of both screens takes less than a few milliseconds in our tests, even for very complex stimuli like random dots and moving gratings. ML2 opts for the benefit of using an independent graphic view and giving the experimenter full-colored, full-featured visual feedback over the additional computational burden. The timing of screen presentation is synchronized with the refresh rate only for the subject's screen, not

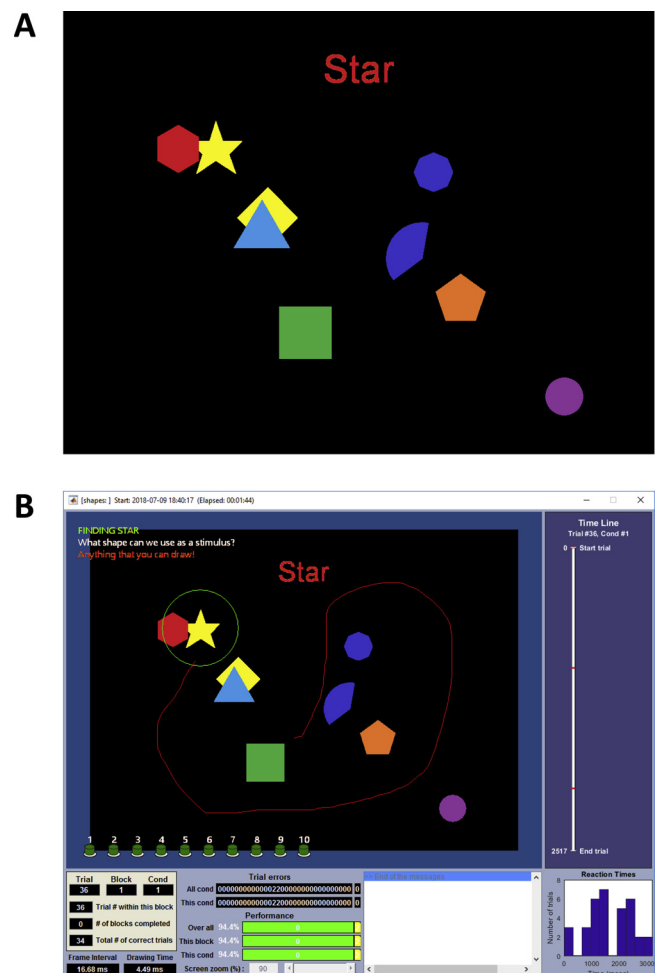


Fig. 5. Screenshots of NIMH MonkeyLogic (ML2). A. The subject's screen. B. The experimenter's screen. MonkeyLogic Graphics Library (MGL) displays visual stimuli on both screens in parallel but overlays task-related information (fixation window, eye trace, button input, user text, etc.) on the experimenter's screen only. It supports transparent colors so that the visual stimuli can be overlapped.

for the experimenter's screen so that updating the experimenter's screen does not affect the timing of visual stimuli.

MGL supports the use of a transparent color (Fig. 5). This feature is useful because an object with a transparent background can be overlapped with other objects without occluding them with its background area. As shown in Fig. 5, multiple objects can be displayed in the same location. In MGL, the transparent color can be set either by specifying a key color that will look transparent or using alpha channel blending. In addition, the shapes that can be used as stimuli are not limited to just circles and rectangles in ML2 (Fig. 5). MGL lets users draw polygons, oval, pie- or any arbitrary shaped stimuli as well as fonts, without creating them as bitmap images. MGL can also create movies directly from files (AVI or MPG) or from MATLAB matrices during intertrial intervals (ITIs) as well as during trial execution. These features enable faster loading and flexible control of animation. For file-sourced movies, MGL supports streaming of the contents so that long movies can be played with no practical limit to length.

Another advantage of MGL is that it is integrated with NIMH DAQ Toolbox at the binary code level and performs certain jobs that require combination play of both components much faster. For example, it is often necessary to send event markers to external devices when presenting stimuli. This kind of job was handled at the slower MATLAB script level in previous tools. In ML2, however, NIMH DAQ Toolbox is called within the same DLL (dynamic linking library) when MGL

Table 1

Resolution of timestamps and latencies of event markers and stimuli. The timestamp resolution is the minimum interval that the tic and toc commands of MATLAB can measure. The latencies of analog stimulations, sounds and photodiode output were measured from the event markers sent out immediately after the stimuli. A negative number indicates that the signal change due to the stimulus was faster than the event marker. All measures were repeated 100 times. The numbers in parentheses are the results with MATLAB R2018a. Because of the just-in-time compilation of the MATLAB execution engine, the first few measures can be slower than the rest. For example, the first six timestamp intervals measured in ML2 with R2018a were 93.81, 15.78, 13.74, 152.55, 176.22 and 1.17 μ s, but the rest of the intervals were all shorter than 1 μ s.

	Original MonkeyLogic (ML1)			NIMH MonkeyLogic (ML2)		
	Min	Median	Max	Min	Median	Max
Timestamp interval (μ s)	7.89	8.18	13.15	5.26 (0.29)	5.55 (0.29)	18.12 (176.22)
Eventmarker (ms)	0.74	0.81	0.93	0.14 (0.17)	0.18 (0.20)	0.31 (0.48)
Stimulation (ms)	−15.68	−9.64	−2.52	−2.75 (−3.05)	−0.05 (−0.05)	−0.03 (−0.03)
Sound, on-board (ms)	302.1	323.13	354.55	34.83 (34.45)	39.92 (39.94)	44.65 (44.73)
Sound, USB (ms)	315.98	326.23	336.6	34.70 (35.45)	40.98 (40.33)	45.08 (46.77)
Photodiode (ms)	0.05	0.15	0.2	−0.05 (−0.07)	−0.03 (−0.03)	0.00 (0.00)

presents stimuli. This tight integration markedly improves the performance of ML2, as shown below.

3.4. Time measure and latencies of stimuli

One measure of behavior control system performance is the time needed to present stimuli and record their timestamps. The precisions of time measurement and stimulus presentation are greatly enhanced in ML2, thanks to NIMH DAQ Toolbox and MGL.

In Windows, the most convenient way to get high-resolution timestamps is to call the QueryPerformanceCounter (QPC) API ([“Acquiring high-resolution time stamps, 2018”](#)). QPC uses the processor’s time stamp counter (TSC) as the basis in Windows 7, Windows Server 2008 R2 and later. The QPC time precision is typically 1024 / (TSC frequency). In our test system, the precision of QPC was 0.29 μ s ($= 1024 / (3.5 \times 10^9 \text{ Hz})$). MATLAB uses QPC to measure elapsed time with the tic and toc commands in version R2006b and later ([Knapp-Cordes and McKeeman, 2011](#)). The resolution of the tic and toc commands suffers from access overhead, but the error was usually below 20 μ s in our tests ([Table 1](#)). This timing precision is sufficient for measurements with millisecond resolution. Therefore, we used QPC-based measures, including the tic and toc commands, to build ML2 and evaluate its performance.

We first tested the time required for an event marker triggered in MonkeyLogic to reach external devices. In neurophysiological experiments, behavioral data and neural data are often collected by two separate systems and synchronized by digital event markers exchanged between systems. For accurate synchronization, the event marker should be transmitted with a very short latency. To estimate this delay, we created a loopback connection between the digital output and analog input of a DAQ board and recorded from the analog input channel while sending out event markers through the digital line. The timestamp of the change in analog input provides a means to measure delay. The median delay between the command execution and the signal change was less than a millisecond in both ML1 and ML2 (0.81 ms and 0.18 ms, respectively), although ML2 was slightly faster

([Table 1](#)).

Next, we measured the latencies of stimuli (stimulation, sound and visual objects) by sending out an event marker simultaneously with the stimuli and calculating the time differences between the event marker output and the signal change at the end devices (analog output channel of the DAQ device, speaker and photodiode). The latencies of the stimuli measured from the event marker were significantly decreased in all stimulus types in ML2 compared to those in ML1 ([Table 1](#)). The median latencies of analog stimulations and visual stimuli were −0.05 ms and −0.03 ms, respectively, which indicates that those stimuli will usually be presented less than 0.18 ms from the moment the command is executed in ML2.

The performance of the sound output is also improved significantly. The median sound latency measured in ML2 was 39.92 ms for the on-board audio codec (40.98 ms for the USB sound adapter; [Table 1](#)). Although this number may not be as fast as those of the other stimuli, it is 8 times faster than the sound latency in ML1. The sound output of ML2 is implemented with Microsoft XAudio2, a low-level, low-latency API intended for developing high performance audio engines for games ([“XAudio2 Introduction, 2018”](#)). There are other audio drivers and APIs that can provide even faster speed, but those do not universally support all computer systems and the improved performance mostly comes from a tradeoff between low latencies and high CPU overheads.

From our latency tests for visual stimuli, we found that achieving a minimum visual latency depends on when the next frame buffer is submitted. Flipping the entire screen at once during the vertical blank time (VB) of the monitor display is a very common technique to avoid graphic objects being torn into two frames ([Sasouvanh and Bazan, 2017](#)). To use this technique, the graphic buffer that contains the next frame needs to be submitted before the VB arrives. If the time window for this buffer submission is missed, the frame will be pushed back to the next VB and the presentation of visual stimuli will be delayed. We tested one CRT monitor and four LCD monitors for the lead time needed to meet the deadline of frame buffer drawing ([Table 2](#)). The time necessary to avoid an added delay varied from monitor to monitor but became longer as the screen resolution increased. For example, a

Table 2

Thresholds for video buffer submission and display lags of the monitors. To be displayed in the next frame, a video buffer should be submitted before the vertical blank. The minimum number of scan lines (“threshold”) that must remain between the current raster position and the vertical blank at the time of buffer submission is shown below for each tested monitor and resolution (in parentheses, equivalent time). Lag is the time of the photodiode signal change following the vertical blank (mean $\pm$ SD).

Model	Type	Resolution	Refresh rate (Hz)	Threshold (Lines)	Lag (msec)
Dell P793	CRT	1024 $\times$ 768	75	40 ($\approx$ 0.69 ms)	0.09 $\pm$ 0.01
3 M C1500SS	LCD	1024 $\times$ 768	75	40 ($\approx$ 0.69 ms)	3.39 $\pm$ 0.01
3 M C2167PW	LCD	1920 $\times$ 1080	60	140 ($\approx$ 2.16 ms)	2.59 $\pm$ 0.04
Dell 2001FP	LCD	1600 $\times$ 1200	60	190 ($\approx$ 2.64 ms)	24.78 $\pm$ 0.03
Dell P2715Q	LCD	3840 $\times$ 2160	60	630 ($\approx$ 4.86 ms)	2.10 $\pm$ 0.03

monitor with 1024×168 resolution (3M C1500SS) displayed the submitted frame immediately after the upcoming VB when the frame was submitted 0.69 ms before the VB. However, a monitor with 3840×2160 resolution (Dell P2715Q) required a lead time of 4.86 ms. Long lead times reduce the amount of time available for data and video processing between frames. ML2 provides a tool (“photodiode tuner”) to choose the minimum necessary lead time. It should be used to maximize the performance of each ML2 installation.

LCD monitors have display lags. Unlike CRT monitors, LCD monitors have a fixed grid of pixels on the screen. If a provided video source is not in the native resolution of the monitors, the monitor will upscale or downscale the source to fit to the pixel grid. Many LCD monitors also buffer and process image data before displaying it to improve image quality. This additional processing is the source of display lag. All LCD monitors we tested had at least a few milliseconds of lag. One had a lag of 24.78 ms, even when the frame buffers were submitted early (Table 2). Since this display lag adds some extra delay to visual stimuli, experiments sensitive to timing of visual stimuli must account for these delays. The ML2 photodiode tuner tool can be used to estimate the display lag of LCD monitors.

3.5. Movie generation and general performance

The general performance of MonkeyLogic was tested with a task in which a random dot motion stimulus was presented as a movie. ML2 can present movies directly from files (AVI and MPG) or from MATLAB matrices created by user functions, called GEN functions. GEN functions are called with different parameters during ITIs or during the task so that the movies can be dynamically generated for each trial. In addition, ML2 provides a way to create stimuli beforehand, store them in the so-called userloop function as persistent variables, and use them repeatedly, which decreases the trial preparation time significantly. Lastly, the scene framework enables ML2 to generate random dot kinematograms on the fly without any stored movie data. We tested all of these methods and compared ML2 performance with the performance of ML1. ML1 also supports movie presentation from AVI files and from GEN functions and allows preloading movie data into MAT files (Asaad et al., 2012). However, it does not support reuse of created movies or online generation of random dot motion.

Table 3 shows the times for executing major functions of MonkeyLogic during the task. The performance of ML2 was similar to or just slightly better than that of ML1 for the functions, such as `toggleobject()`, `eyejoytrack()` and `eventmarker()`. A dramatic difference was observed during the trial entry and exit, where trial initialization and data storing occurs. For example, ML2 required 22 ms for the trial entry and exit together when the movie was created from a file source, which was 9 times faster than the same procedure measure with ML1 (199 ms). This improvement can be traced to the new object-oriented implementation of ML2, which speeds function initialization and data storage.

The performance of creating movies is also improved in ML2 (Table 3). In these tests the stimuli were generated during the ITI period causing the length of ITI to vary depending on the method of movie creation. When the random dot motion movies were loaded from the files and from a GEN script, the ITI periods were 159.65 ms and 163.61 ms, respectively. These periods were about an order of magnitude shorter than ML1 under the same conditions (1895.38 ms and 1660.10 ms, respectively). ML2 was faster than ML1 even when ML1 loaded the movies from preprocessed MAT files. The userloop function and the scene framework of ML2 provided additional time savings. With the userloop function, ML2 did not need to recreate the same movies for each trial, so the ITI was greatly reduced (66.26 ms). The RandomDotMotion adapter of ML2 was used to draw the stimulus online in the scene framework version of the task implementation. This adapter just moved small dots frame by frame and did not create movie data to cover the whole aperture of the stimulus. Therefore, its stimulus preparation time (92.47 ms) was also much shorter than the first two

movie-based methods.

4. Discussion

We developed ML2 to build on the success of ML1 and support the latest (64-bit) MATLAB computing environment. Several key components and features were added to solve earlier issues and improve the performance of the software. NIMH DAQ Toolbox fixed the problem with the subpar sample readout intervals of MATLAB DAQ Toolbox, enabled millisecond-resolution behavior tracking with only one DAQ device and thus reduced the hardware cost for a MonkeyLogic system. It also supports a variety of input devices including USB-type DAQ boards and other digital devices that do not interface with DAQ boards, such as touchscreens, TCP/IP eye trackers, USB joysticks and webcams. MGL provides a full-colored, information-rich behavior monitoring environment that fully utilizes the power of modern graphics hardware. It allows the use of transparent colors, arbitrarily shaped figures and lengthy movies so that a wide range of visual stimuli can be used. The binary-level integration of NIMH DAQ Toolbox and MGL significantly improved the synchronization of event markers and stimuli and hence the temporal precision of data recording. Further, the new scripting framework allows users of ML2 to create dynamic, behavior-responsive stimuli in real-time and handle a wider range of complex behavioral requirements.

ML2 currently requires Windows (Windows 7 Service Pack 1 or later) and MATLAB (R2011a or later), which are not free. We chose the Windows platform for hardware and driver back-compatibility with existing ML1 installations. This choice leveraged the preexisting user base and likely led to the rapid adoption of ML2. We also considered other open-source MATLAB equivalents, such as GNU Octave. However, these platforms currently lack sufficient support for the graphic user interface and object-oriented programming features heavily leveraged to implement much of ML2’s functionality. We continue to monitor the development of MATLAB alternatives running on Windows or other operating systems in the event that an alternative might be more affordable and still widely adopted.

ML2’s two main components, NIMH DAQ Toolbox and MGL, are both in-house developed with the C++ language. These custom software components provide more complete control over the MonkeyLogic system. For example, ML2 supports devices not supported by the MATLAB interface, and improves the performance by moving frequently used functions from MATLAB scripts to faster C++ code. Fixing the DAQ problem would not have been possible if constrained by a proprietary solution. Importantly, the tools can be revised with our source code for future changes in Windows or MATLAB, staving off obsolescence for a longer period than ML1. NIMH DAQ Toolbox currently supports only National Instruments products. This is not a limitation of the software design and expanded hardware support is on our roadmap for future development.

MGL is comparable to another MATLAB-based graphics tool, Psychophysics Toolbox (Brainard, 1997; Pelli, 1997; <http://psychtoolbox.org>). Both are developed to present visual or auditory stimuli in a time sensitive manner. Psychophysics Toolbox supports more platforms and more complex manipulations for visual stimuli. However, ML2 provides much greater flexibility for complex experimental paradigms. Psychophysics Toolbox is primarily a wrapper of low-level video functions and makes users responsible for coding task events and flows. ML2 provides a trial-based framework and handles creation and destruction of stimuli, trial selection and randomization (when necessary), data acquisition synchronized to behavioral events (via NIMH DAQ Toolbox) and creation of data files. These features are of central value to the intended users of ML2, supporting their complex task designs and insulating them, somewhat, from hardware control and timing accuracy considerations.

The latency of the sound production was significantly decreased in ML2 relative to ML1 (median: 39.92 ms vs. 323.13 ms on the on-board

Table 3
Execution time of MonkeyLogic functions. The measured times (mean $\pm$ SD) are in milliseconds and based on 100 trials of the performance test (see Methods), including the first trial. The numbers in parentheses are the results with MATLAB R2018a. The “Entry” time refers to the amount of time required for initialization of each function, before the execution of the core activity. The “Core” time is the amount of time required to execute the essential activity of the function. The “Exit” time reflects the amount of time required to clean up and leave the function, after the core activity has completed. For the “Trial” row, the entry time corresponds to the time required to initialize all the sub-functions and analog data acquisition and the exit time is the time required to store collected data to the disk. The intertrial interval includes the time required to update behavior performance measures on the experimenter’s screen but most of it is the time required to load stimuli. The eventmarker function of ML2 is a one-line function, so there is no entry or exit time. The adapters of ML2 are executed with `run_scene()`, instead of `toggleobject()` and `eyejoytrack()`.

Function	Original MonkeyLogic (ML1)			NIMH MonkeyLogic (ML2)			
	mov	mov (preprocessed)	gen	mov	gen	userloop (preloading)	adapter
Trial							
Entry	107.60 $\pm$ 1.91	108.18 $\pm$ 0.51	107.25 $\pm$ 0.95	4.23 $\pm$ 0.21 (4.67 $\pm$ 3.89)	4.21 $\pm$ 0.18 (4.18 $\pm$ 1.90)	4.23 $\pm$ 0.13 (4.11 $\pm$ 1.65)	5.33 $\pm$ 0.51 (4.79 $\pm$ 4.94)
Exit	90.57 $\pm$ 19.20	92.87 $\pm$ 17.37	89.43 $\pm$ 20.88	17.43 $\pm$ 0.41 (17.59 $\pm$ 2.19)	17.41 $\pm$ 0.36 (17.51 $\pm$ 1.29)	17.34 $\pm$ 0.38 (17.35 $\pm$ 1.38)	17.16 $\pm$ 0.39 (17.30 $\pm$ 2.59)
Toggle Object							
Entry	0.38 $\pm$ 0.01	0.36 $\pm$ 0.01	0.37 $\pm$ 0.00	0.33 $\pm$ 0.01 (0.40 $\pm$ 0.02)	0.33 $\pm$ 0.01 (0.40 $\pm$ 0.01)	0.33 $\pm$ 0.01 (0.40 $\pm$ 0.01)	
Core	0.37 $\pm$ 0.03	0.37 $\pm$ 0.02	0.36 $\pm$ 0.02	0.12 $\pm$ 0.00 (0.06 $\pm$ 0.00)	0.12 $\pm$ 0.00 (0.06 $\pm$ 0.00)	0.12 $\pm$ 0.00 (0.06 $\pm$ 0.00)	
Exit	0.04 $\pm$ 0.00	0.04 $\pm$ 0.00	0.04 $\pm$ 0.00	0.34 $\pm$ 0.01 (0.33 $\pm$ 0.02)	0.34 $\pm$ 0.01 (0.33 $\pm$ 0.01)	0.34 $\pm$ 0.01 (0.33 $\pm$ 0.01)	
EyeJoyTrack							
Entry	1.03 $\pm$ 0.05	1.06 $\pm$ 0.07	1.05 $\pm$ 0.06	0.27 $\pm$ 0.02 (0.23 $\pm$ 0.05)	0.28 $\pm$ 0.02 (0.22 $\pm$ 0.04)	0.27 $\pm$ 0.02 (0.22 $\pm$ 0.04)	
Core	0.47 $\pm$ 0.02	0.45 $\pm$ 0.00	0.45 $\pm$ 0.00	0.54 $\pm$ 0.00 (0.25 $\pm$ 0.01)	0.54 $\pm$ 0.00 (0.25 $\pm$ 0.00)	0.54 $\pm$ 0.00 (0.25 $\pm$ 0.00)	
Exit	0.81 $\pm$ 0.06	0.81 $\pm$ 0.06	0.81 $\pm$ 0.06	0.28 $\pm$ 0.03 (0.30 $\pm$ 0.04)	0.28 $\pm$ 0.03 (0.30 $\pm$ 0.03)	0.28 $\pm$ 0.03 (0.31 $\pm$ 0.03)	
EventMarker							
Entry	0.02 $\pm$ 0.00	0.02 $\pm$ 0.00	0.02 $\pm$ 0.00	0.35 $\pm$ 0.01 (0.31 $\pm$ 0.10)	0.35 $\pm$ 0.01 (0.31 $\pm$ 0.04)	0.35 $\pm$ 0.01 (0.31 $\pm$ 0.04)	0.34 $\pm$ 0.01 (0.31 $\pm$ 0.09)
Core	0.56 $\pm$ 0.06	0.59 $\pm$ 0.04	0.55 $\pm$ 0.03				
Exit	0.02 $\pm$ 0.00	0.02 $\pm$ 0.00	0.02 $\pm$ 0.00				
RunScene							
Entry							0.79 $\pm$ 0.02 (0.29 $\pm$ 0.10)
Analyze							0.89 $\pm$ 0.01 (0.37 $\pm$ 0.01)
Draw							1.01 $\pm$ 0.01 (0.86 $\pm$ 0.01)
Exit							0.71 $\pm$ 0.02 (0.45 $\pm$ 0.13)
Intertrial Interval							
Preparation	1895.38 $\pm$ 63.82	542.11 $\pm$ 1.00	1660.10 $\pm$ 15.54	159.65 $\pm$ 38.66 (271.19 $\pm$ 63.56)	163.61 $\pm$ 5.21 (257.29 $\pm$ 12.84)	66.26 $\pm$ 4.30 (167.36 $\pm$ 9.69)	92.47 $\pm$ 20.70 (198.57 $\pm$ 30.81)

audio codec) but was not as short as those of analog stimulations or visual stimuli (< 1 ms). It is not likely that this latency will cause asynchrony perception when a sound track is presented simultaneously with a corresponding video in ML2. According to human audiovisual asynchrony detection studies (Maier et al., 2011; Miller and D'Esposito, 2005; Stevenson et al., 2010), the temporal window in which perceptual fusion occurs is skewed toward the direction where the video leads the audio and the fusion thresholds typically reported exceeds the sound latency of ML2. This relatively large sound latency nevertheless may be unacceptable for some experiments. Unfortunately, the production rate of PC sound output is limited by the signal processing and mixing that occurs during sound rendering (Marshall and Hill, 2017). If zero-latency sounds are necessary, one should bypass the audio architecture of Windows. This can be done using the analog output channels of a DAQ device, using an arbitrary waveform generator, or using a specialized audio device.

ML2 is designed to run the tasks developed for ML1 seamlessly with no or the minimum modification. Those already familiar with ML1 should be able to maintain existing projects with minimal changes. (Those interested in the ML1-style task coding may see Asaad and Eskandar, 2008a) For those who are new to MonkeyLogic, documents and code examples are available with the ML2 distribution package (<https://monkeylogic.nimh.nih.gov/>). ML2 is still in active development and there is also an online forum on the NIMH MonkeyLogic website for users to exchange questions and programming tips.

Declarations of interest

None.

Acknowledgements

We thank Drs. Wael Asaad and David Freedman for their support throughout the development of NIMH MonkeyLogic. This research was supported by the Intramural Research Program of the National Institute

of Mental Health, National Institutes of Health (EAM, ZIAMH002886).

References

- Acquiring high-resolution time stamps, 2018. Microsoft Docs. (Accessed 12.6.18). <https://docs.microsoft.com/en-us/windows/desktop/SysInfo/acquiring-high-resolution-time-stamps>.
- Asaad, W.F., Eskandar, E.N., 2008a. A flexible software tool for temporally-precise behavioral control in Matlab. *J. Neurosci. Methods* 174, 245–258. <https://doi.org/10.1016/j.jneumeth.2008.07.014>.
- Asaad, W.F., Eskandar, E.N., 2008b. Achieving behavioral control with millisecond resolution in a high-level programming environment. *J. Neurosci. Methods* 173, 235–240. <https://doi.org/10.1016/j.jneumeth.2008.06.003>.
- Asaad, W.F., Santhanam, N., McClellan, S.M., Freedman, D.J., 2012. High-performance execution of psychophysical tasks with complex visual stimuli in MATLAB. *J. Neurophysiol.* 109, 249–260. <https://doi.org/10.1152/jn.00527.2012>.
- Brainard, D.H., 1997. The psychophysics toolbox. *Spat. Vis.* 10, 433–436.
- Knapp-Cordes, M., McKeeman, B., 2011. Improvements to tic and toc Functions for Measuring Absolute Elapsed Time Performance in MATLAB. (Accessed 12.6.18). <https://www.mathworks.com/company/newsletters/articles/improvements-to-tic-and-toc-functions-for-measuring-absolute-elapsed-time-performance-in-matlab.html>.
- Laboratory of Neuropsychology / NIMH / NIH, 1989. NIMH Cortex. <ftp://helix.nih.gov/lsn/cortex/> (Accessed 12.6.18).
- Maier, J.X., Di Luca, M., Noppeney, U., 2011. Audiovisual asynchrony detection in human speech. *J. Exp. Psychol. Hum. Percept. Perform.* 37, 245–256. <https://doi.org/10.1037/a0019952>.
- Marshall, D., Hill, A., 2017. Low Latency Audio. Microsoft Docs. (Accessed 12.6.18). <https://docs.microsoft.com/en-us/windows-hardware/drivers/audio/low-latency-audio>.
- Miller, L.M., D'Esposito, M., 2005. Perceptual fusion and stimulus coincidence in the cross-modal integration of speech. *J. Neurosci.* 25, 5884–5893. <https://doi.org/10.1523/JNEUROSCI.0896-05.2005>.
- Pelli, D.G., 1997. The VideoToolbox software for visual psychophysics: transforming numbers into movies. *Spat. Vis.* 10, 437–442.
- Rosenfeld, S.A., 1983. Laboratory Instrument Computer (LINC) "The Genesis of a Technological Revolution". <https://history.nih.gov/exhibits/linc/> (Accessed 12.6.18).
- Sasouvanh, M., Bazan, N., 2017. Tearing. Microsoft Docs. (Accessed 12.6.18). <https://docs.microsoft.com/en-us/windows-hardware/drivers/display/tearing>.
- Stevenson, R.A., Altieri, N.A., Kim, S., Pisoni, D.B., James, T.W., 2010. Neural processing of asynchronous audiovisual speech perception. *Neuroimage* 49, 3308–3318. <https://doi.org/10.1016/j.neuroimage.2009.12.001>.
- XAudio2 Introduction. 2018. Microsoft Docs. <https://docs.microsoft.com/en-us/windows/desktop/xaudio2/xaudio2-introduction> (Accessed 12.6.18).